

Voici concrètement comment les variables de **adaptiveScore** sont remplies dans le code réel d'**AdaptivePathService**.

Formule réelle :

```
adaptiveScore =  
0.35 × prerequisiteScore  
+ 0.25 × diagnosticWeaknessScore  
+ 0.15 × historicalPerformanceScore  
+ 0.15 × pedagogicalOrderScore  
+ 0.10 × engagementScore
```

1. prerequisiteScore

Objectif : mesurer si les prérequis du concept sont satisfaits.

Logique réelle :

```
Si le concept a des prérequis manquants :  
    prerequisiteScore = 0.0  
  
Sinon si le concept possède des prérequis déclarés et qu'ils sont satisfaits :  
    prerequisiteScore = 1.0  
  
Sinon, aucun prérequis bloquant déclaré :  
    prerequisiteScore = 0.5
```

Exemples :

Situation	Valeur
Concept bloqué car prérequis non maîtrisé	0.0
Concept accessible avec prérequis satisfaits	1.0
Concept sans prérequis déclaré	0.5

Dans ton scénario :

Variables -> Conditions

Comme **Variables** est maîtrisé, **Conditions** a ses prérequis satisfaits :

```
prerequisiteScore = 1.0
```

2. diagnosticWeaknessScore

Objectif : donner plus d'importance aux concepts faibles détectés par le diagnostic.

Logique réelle :

```
Si le concept lui-même est échoué dans le diagnostic :  
    diagnosticWeaknessScore = 1.0  
  
Sinon si un prérequis manquant du concept est échoué dans le diagnostic :  
    diagnosticWeaknessScore = 0.7  
  
Sinon :  
    diagnosticWeaknessScore = 0.3
```

Exemples :

Situation	Valeur
Concept échoué dans le diagnostic	1.0
Prérequis proche échoué	0.7
Pas de faiblesse diagnostic directe	0.3

Dans ton scénario observé, **Conditions** a :

```
"diagnosticWeaknessScore": 0.3
```

Donc le moteur ne l'a pas considéré comme faiblesse directe dans le score. La difficulté sur **Conditions** apparaît plutôt ensuite dans le PLP via :

```
TO_REVIEW  
persistentDifficulty = true  
repeatedFailuresCount = 3
```

3. historicalPerformanceScore

Objectif : intégrer les performances passées de l'apprenant.

Logique réelle :

```
Si aucune trace :  
    historicalPerformanceScore = 0.5  
  
Sinon calculer la moyenne des scoreObtenu.
```

```
Si moyenne >= 80 :  
    historicalPerformanceScore = 1.0  
  
Sinon si moyenne >= 60 :  
    historicalPerformanceScore = 0.7  
  
Sinon :  
    historicalPerformanceScore = 0.4
```

Exemples :

Moyenne des traces	Valeur
Aucune trace	0.5
Moyenne ≥ 80	1.0
Moyenne entre 60 et 79	0.7
Moyenne < 60	0.4

Dans ton scénario :

Après diagnostic :

```
score diagnostic = 62.5  
historicalPerformanceScore = 0.7
```

Après échec formatif sur Conditions :

```
scores = 62.5 et 45  
moyenne = 53.75  
historicalPerformanceScore = 0.4
```

4. pedagogicalOrderScore

Objectif : respecter l'ordre pédagogique du cours.

Logique réelle :

```
pedagogicalOrderScore = 1 - positionConcept / nombreTotalConcepts
```

avec un minimum de :

0.1

Puis le score est arrondi.

Exemple avec 5 concepts :

Concept	Position	Calcul	Valeur
Variables	0	$1 - 0/5$	1.0
Conditions	0 ou 1 selon ordre interne aplati	proche de 1.0	
Boucles	plus loin	diminue	
Fonctions	plus loin	diminue	
Tableaux	dernier	plus faible	

Dans ta réponse API, **Conditions** a :

```
"pedagogicalOrderScore": 1.0
```

Donc le moteur considère **Conditions** comme un concept prioritaire dans l'ordre pédagogique actuel.

5. engagementScore

Objectif : approximer l'engagement récent de l'apprenant à partir des traces et labs.

Logique réelle :

```
Si beaucoup de labs commencés mais non terminés
ou beaucoup d'échecs :
```

```
    engagementScore = 0.3
```

```
Sinon si lab récent complété :
```

```
    engagementScore = 1.0
```

```
Sinon si trace récente :
```

```
    engagementScore = 0.7
```

```
Sinon si au moins un lab complété :
```

```
    engagementScore = 1.0
```

```
Sinon si au moins une trace existe :
```

```
    engagementScore = 0.7
```

```
Sinon :
```

```
    engagementScore = 0.5
```

Exemples :

Situation	Valeur
Aucune activité	0.5
Trace récente	0.7
Lab complété	1.0
Plusieurs échecs / faible engagement	0.3

Dans ton scénario, après diagnostic puis échec formatif, il existe des traces récentes :

```
"engagementScore": 0.7
```

Exemple réel : Conditions après diagnostic

Valeurs observées :

```
{
  "prerequisiteScore": 1.0,
  "diagnosticWeaknessScore": 0.3,
  "historicalPerformanceScore": 0.7,
  "pedagogicalOrderScore": 1.0,
  "engagementScore": 0.7
}
```

Calcul :

```
adaptiveScore =
0.35 × 1.0
+ 0.25 × 0.3
+ 0.15 × 0.7
+ 0.15 × 1.0
+ 0.10 × 0.7

= 0.35
+ 0.075
+ 0.105
+ 0.15
+ 0.07

= 0.75
```

Résultat API :

```
"adaptiveScore": 0.75
```

Exemple réel : Conditions après échec formatif

Valeurs observées :

```
{
  "prerequisiteScore": 1.0,
  "diagnosticWeaknessScore": 0.3,
  "historicalPerformanceScore": 0.4,
  "pedagogicalOrderScore": 1.0,
  "engagementScore": 0.7
}
```

Calcul :

```
adaptiveScore =
0.35 × 1.0
+ 0.25 × 0.3
+ 0.15 × 0.4
+ 0.15 × 1.0
+ 0.10 × 0.7

= 0.35
+ 0.075
+ 0.06
+ 0.15
+ 0.07

= 0.705
≈ 0.71
```

Résultat API :

```
"adaptiveScore": 0.71
```

Résumé simple :

Variable	Ce qu'elle mesure	Source
prerequisiteScore	prérequis satisfaits ou non	graphe Neo4j
diagnosticWeaknessScore	lacune issue du diagnostic	dernière trace diagnostic
historicalPerformanceScore	moyenne des scores passés	tracking-service

Variable	Ce qu'elle mesure	Source
pedagogicalOrderScore	position dans l'ordre pédagogique	graphe du cours
engagementScore	activité récente / labs / traces	tracking-service

Donc le score adaptatif est un score rule-based explicable, calculé à partir du graphe de compétences, des traces d'apprentissage, du diagnostic et de l'ordre pédagogique.